

Competitive Programming

Section 36: Fenwick Trees

Competitive Programming Course

2026-08-27

Table of contents

1	Fenwick trees without black boxes	3
1.1	Why choose exactly these blocks?	3
1.2	The picture to keep in mind	4
1.3	Building intuition with one update	5
2	Coordinate compression with Fenwick	6
2.1	The problem: values are not usable indices	6
2.2	Step 1: build the value-to-rank map	6
2.3	Step 2: translate the array	6
2.4	Step 3: decide what the Fenwick tree stores	7
2.5	A short inversion trace	7
2.6	Implementation recipe	7
3	Difference arrays and Fenwick	8
4	Checklist	9
5	Minimal templates	10
6	Local exercises	11
7	How to use the checker	12
8	Contest decisions, invariants, and traces	13
8.1	Trace: point add and range sum	13
8.2	Trace: inversions with duplicates	13
8.3	Difference trace	13
9	Language notes and proof habits	14
9.1	Formative checks	14
10	Submission workflow	15

1 Fenwick trees without black boxes

A Fenwick tree, also called a binary indexed tree, stores prefix information in an array `bit[1..n]`. The key idea is that index `i` owns a block whose length is the value of its lowest set bit:

```
lowbit(i) = i & -i

bit[i] stores the sum of:
i-lowbit(i)+1, ..., i
```

1.1 Why choose exactly these blocks?

Start from the operation we want: `prefix(r)` must add all values from 1 to `r`, while a point update must avoid rewriting every later prefix. Storing all prefix sums makes queries instant but updates linear. Storing only individual values makes updates instant but queries linear. A Fenwick tree keeps a small set of reusable, power-of-two blocks between those two extremes.

Imagine building those blocks in layers:

```
Every index owns its final 1 element.
Every multiple of 2 can summarize its final 2 elements.
Every multiple of 4 can summarize its final 4 elements.
Every multiple of 8 can summarize its final 8 elements.
...
```

At index `i`, keep the largest block from this list that ends at `i`. Thus:

odd <code>i</code>	owns 1 element
<code>i</code> divisible by 2, not 4	owns 2 elements
<code>i</code> divisible by 4, not 8	owns 4 elements
<code>i</code> divisible by 8, not 16	owns 8 elements

The largest power of two dividing `i` is exactly `lowbit(i)`. In binary, it is the value represented by the rightmost 1 bit:

<code>i</code>	binary	lowbit	block ending at <code>i</code>
3	0011	1	[3,3]
4	0100	4	[1,4]
6	0110	2	[5,6]
8	1000	8	[1,8]

This alignment is what makes the blocks fit together. To answer `prefix(r)`, take the largest stored block ending at `r`. The still-missing part ends just before that block, at `r - lowbit(r)`,

so repeat there. Each subtraction clears the rightmost 1 bit of r . For $r = 13$, whose binary form is 1101, the block sizes are 1, then 4, then 8; they cover the prefix without gaps or overlap.

A point update walks the same hierarchy in reverse. If position i changes, update the block ending at i , then the next larger aligned block containing that position. The move $i += \text{lowbit}(i)$ performs the necessary binary carry to reach its next owner. For position 3, that gives $3 \rightarrow 4 \rightarrow 8$.

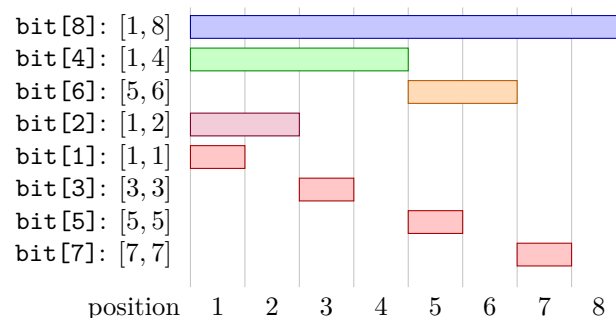
So the Fenwick layout is not an arbitrary trick: it is a compact hierarchy of aligned power-of-two suffixes. Binary representation tells us both how to decompose a prefix and how to find every stored block affected by an update.

Example for $n = 8$:

i	$\text{lowbit}(i)$	covered segment
1	1	[1,1]
2	2	[1,2]
3	1	[3,3]
4	4	[1,4]
5	1	[5,5]
6	2	[5,6]
7	1	[7,7]
8	8	[1,8]

1.2 The picture to keep in mind

For $n = 8$, this is the whole structure. Each horizontal bar is one entry of `bit`; it owns the highlighted interval, not just the value at its index. The bars overlap on purpose.



For example, position 3 belongs to exactly the three block owners that an update visits: `bit[3]`, `bit[4]`, and `bit[8]`. Conversely, `prefix(6)` reads the two disjoint bars `bit[6]` ([5,6]) and `bit[4]` ([1,4]), which together cover [1,6] exactly once.

To compute a prefix sum `sum(i)`, repeatedly take the block ending at i , then jump to the index just before that block:

```
answer += bit[i]
i -= i & -i
```

This decomposes $[1, i]$ into disjoint power-of-two blocks. For $i = 13$, the visited indices are:

```
13 -> 12 -> 8 -> 0
[13,13] + [9,12] + [1,8]
```

To add `delta` to position `i`, every Fenwick block containing `i` must increase by `delta`. Those block owners are found by moving upward:

```
bit[i] += delta
i += i & -i
```

Both loops touch at most one index per binary digit, so each operation costs $O(\log n)$.

1.3 Building intuition with one update

Suppose `n = 8` and we add `+5` at position `3`.

The update visits:

```
3 -> 4 -> 8
```

Those are exactly the Fenwick blocks that contain position `3`:

```
bit[3] covers [3,3]
bit[4] covers [1,4]
bit[8] covers [1,8]
```

No other block ending at another index includes position `3`, so no other Fenwick entry changes.

For `sum(6)`, the query visits:

```
6 -> 4 -> 0
```

and reads:

```
bit[6] covers [5,6]
bit[4] covers [1,4]
```

Together these blocks form `[1,6]` without overlap.

2 Coordinate compression with Fenwick

2.1 The problem: values are not usable indices

A Fenwick tree is an array, so its indices must be small positive integers. But values in a problem may be negative, sparse, or as large as 10^9 :

```
a = [50, -10, 50, 900]
```

We cannot access `bit[-10]`, and allocating `bit[1..900]` for only three different values wastes space. For order-based problems, however, the exact distances between values do not matter. We only need to know which value is smaller, equal, or larger.

Coordinate compression creates one dense Fenwick index for each distinct value.

2.2 Step 1: build the value-to-rank map

Copy the values, sort them, and remove duplicates:

```
sorted distinct values = [-10, 50, 900]
```

Number that sorted list starting from 1:

value	-10	50	900
rank	1	2	3

The rank of a value is therefore one plus the number of distinct values smaller than it. Equal values look up the same rank.

2.3 Step 2: translate the array

Use the map on each original value:

original value	50	-10	50	900
compressed rank	2	1	2	3

Compression preserves order and equality:

```
x < y   iff  rank(x) < rank(y)
x = y   iff  rank(x) = rank(y)
```

It does not preserve distances: ranks 2 and 3 differ by one even though $900 - 50 = 850$. That is fine for counting smaller or larger values, but not for calculations that need the original numeric differences.

2.4 Step 3: decide what the Fenwick tree stores

In inversion counting, scan the array from left to right. The scan still uses the original array positions. The Fenwick index is different: it is the value's compressed rank.

Store a frequency at each rank:

```
add(rank(x), 1) means: one more value x has been seen
prefix(k)       means: how many seen values have rank at most k
```

For $x = 50$, whose rank is 2:

```
prefix(1) = number of previous values < 50
prefix(2) = number of previous values <= 50
```

Including rank 2 counts earlier values equal to 50. Therefore the number of earlier values strictly greater than the current value is:

```
number_seen - prefix(rank(current_value))
```

2.5 A short inversion trace

Process $[50, -10, 50, 900]$, whose ranks are $[2, 1, 2, 3]$:

current	seen before	count <= current	count > current
50	$[]$	0	0
-10	$[50]$	0	1
50	$[50, -10]$	2	0
900	$[50, -10, 50]$	3	0

The total is one inversion: $(50, -10)$.

2.6 Implementation recipe

C++:

```
vector<long long> values = a;
sort(values.begin(), values.end());
values.erase(unique(values.begin(), values.end()), values.end());

int rank = int(lower_bound(values.begin(), values.end(), x) - values.begin()) + 1;
```

Python:

```
values = sorted(set(a))
rank = {value: index + 1 for index, value in enumerate(values)}
```

The $+1$ is essential because Fenwick trees use 1-based indices.

3 Difference arrays and Fenwick

A difference array stores the change when moving one position right. Define $a[0] = 0$, then:

```
diff[1] = a[1]
diff[i] = a[i] - a[i-1]
a[i] = diff[1] + ... + diff[i]
```

For example:

index	1	2	3	4	5
a	10	20	30	40	50
diff	10	10	10	10	10

The prefix sums of `diff` reconstruct `a`:

```
10, 10+10, 10+10+10, ... = 10, 20, 30, 40, 50
```

Now add +7 to the range [2,4]. The desired new array is:

index	1	2	3	4	5
new a	10	27	37	47	50
new diff	10	17	10	10	3

Only two differences changed:

```
diff[2] += 7    # begin carrying +7 from position 2 onward
diff[5] -= 7    # stop carrying +7 after position 4
```

Why does this work? Every prefix sum before the left endpoint sees neither change. A prefix sum from the left endpoint through the right endpoint sees $+x$ but not $-x$, so the value increases by x . A prefix sum after the right endpoint sees both changes, whose total is zero. Thus a range update becomes two point updates.

If a Fenwick tree stores `diff`, it can apply those two point updates in $O(\log n)$. The current value at position i is `prefix(i)`, also in $O(\log n)$. This is the range-add / point-query variant; it is different from the earlier point-add / range-sum variant, where the Fenwick tree stores `a` itself.

4 Checklist

```
Use 1-based indices internally.  
Decide whether the tree stores values or differences.  
For range sums, answer  $\text{sum}(r) - \text{sum}(l-1)$ .  
For inversions, count strictly greater values; duplicates are not inversions.  
Use 64-bit integers for counts and sums.
```

5 Minimal templates

C++:

```
struct Fenwick {
    int n;
    vector<long long> bit;
    Fenwick(int n) : n(n), bit(n + 1, 0) {}
    void add(int i, long long delta) {
        for (; i <= n; i += i & -i) bit[i] += delta;
    }
    long long sum(int i) const {
        long long ans = 0;
        for (; i > 0; i -= i & -i) ans += bit[i];
        return ans;
    }
    long long range_sum(int l, int r) const {
        return sum(r) - sum(l - 1);
    }
};
```

Python:

```
class Fenwick:
    def __init__(self, n):
        self.n = n
        self.bit = [0] * (n + 1)

    def add(self, i, delta):
        while i <= self.n:
            self.bit[i] += delta
            i += i & -i

    def sum(self, i):
        ans = 0
        while i > 0:
            ans += self.bit[i]
            i -= i & -i
        return ans
```

6 Local exercises

- A. Point Add Range Sum: Maintain an array. Operation 1 i x adds x to $a[i]$. Operation 2 l r asks for the sum of $a[l..r]$.
- B. Inversion Count: Given an array, count pairs $i < j$ with $a[i] > a[j]$. Values may repeat and may be large.
- C. Range Add Point Query: Maintain an array. Operation 1 l r x adds x to every element in $[l, r]$. Operation 2 i asks for the current value of $a[i]$.
- D. Offline Distinct Range Queries: For each static range $[l, r]$, count its distinct values. The queries may be reordered; attempt this blind before reading the editorial.

7 How to use the checker

The local checker compiles/runs both languages and compares output tokens exactly:

```
CP_TARGET=solution python3 sections/36_fenwick_trees/check.py
```

Replace `CP_TARGET=solution` by your own `solve.cpp` or `solve.py` attempts when practicing.
Exercises

Use A for point-add/range-sum indexing, B to count inversions after coordinate compression, and C to turn range additions into a difference Fenwick tree. Use D as the blind synthesis problem: decide what each active Fenwick position must mean while sweeping offline queries. Test index 1, the final index, duplicate values, and queries supplied in an inconvenient order.

8 Contest decisions, invariants, and traces

Do not choose a Fenwick tree because a problem says “array.” Choose it when the operations are additions and prefix aggregates, and both updates and queries must be fast. A plain prefix-sum array makes queries $O(1)$ but a point update $O(n)$; direct summation makes an update $O(1)$ but each range query $O(n)$. A Fenwick tree gives both in $O(\log n)$.

The invariant is precise: after any sequence of `add` calls, `bit[i]` equals the sum of the original logical values in $[i - \text{lowbit}(i) + 1, i]$. Prefix query visits disjoint blocks that partition $[1, i]$; update visits exactly the block owners whose block contains the changed position. These are the two facts to prove before trusting the loops.

8.1 Trace: point add and range sum

Start with `a=[2,1,4,3,5]`. Build by calling `add(i,a[i])`. For `add(3,+6)`, the logical array becomes `[2,1,10,3,5]`; the visited owners are `3,4,8,...`. For query `[2,4]`, compute `prefix(4)-prefix(1)=(2+1+10+3)-2=14`. Never try to make `bit[3]` mean “the value at index 3”; it is a block sum.

8.2 Trace: inversions with duplicates

For `[3,1,2,2]`, compress to ranks `[3,1,2,2]`. Read left to right. At the third item, `seen=2` and `prefix(rank 2)=1`, so exactly one prior value is greater. At the final 2, the prefix count is two, so again one value is greater. Equal values are included in the prefix and are therefore not counted as inversions. The final answer is three.

8.3 Difference trace

For `a=[10,20,30,40]`, store differences `[10,10,10,10]`. Adding `+5` on `[2,3]` changes only `diff[2] += 5` and `diff[4] -= 5`, giving `[10,15,10,5]`. Its prefix sums reconstruct `[10,25,35,40]`. This is why a single Fenwick over differences solves range-add/point-query.

9 Language notes and proof habits

Use 1-based internal indices: `lowbit(0)=0`, so an update starting at zero loops forever. In C++, use `long long` for sums and inversion counts. In Python, the integer size is safe, but buffered input and collecting answers before one final print matter for large operation counts. Coordinate compression must use `sorted(set(a))`, not `sorted(a)`, so equal values share a rank.

Before coding, say aloud which quantity `prefix(i)` represents. For normal storage it is `a[1]+...+a[i]`; for difference storage it is the *current value* at position `i`. This one sentence prevents most range/update mismatches.

Wrong approach	Why it fails	Test
use index 0	<code>lowbit</code> never advances	update/query at first position
use <code>seen-prefix(rank-1)</code> for inversions	counts equal values as greater	<code>[2,2]</code>
omit <code>r+1</code> subtraction	update leaks past the range	update <code>[2,3]</code> , query 4
store sums in <code>int</code>	totals can exceed 32 bits	many large additions

9.1 Formative checks

1. Which blocks does `prefix(13)` read? Hint: repeatedly subtract `lowbit`. Answer: owners 13,12,8, covering `[13,13]`, `[9,12]`, `[1,8]`.
2. Why is `previous - prefix(rank)` the inversion contribution? Answer: it removes every previous value $\leq$ the current value, leaving exactly $>$.
3. What two difference entries change for add -7 on `[1,r]`? Answer: `diff[1]-=7`, `diff[r+1]+=7`.

10 Submission workflow

Use a slow array model for stress tests: perform each update literally and compute each query by a loop. Compare it with the Fenwick result on small random arrays, especially negative additions, index one, index n , repeated values, and a full-range update. Then run:

```
CP_TARGET=solution python3 sections/36_fenwick_trees/check.py
```